

Reinforcement Learning: Markov Decision Processes (ESE 3990)

Instructor:
Antonio Loquercio

Fall 2026
University of Pennsylvania

Summary of today's class

- Definitions
 - Markov Processes
 - Markov Reward Processes
 - Markov Decision Processes
 - Value Functions
 - Bellman Equation
 - Graph Representations
 - Extension to MDPs
- Readings & Sources:
 - Sutton and Barto [Ch. 3];
 - David Silver [Intro to RL, MDPs]

Introduction to MDPs

- **Markov decision processes** formally describe the *interaction* between the agent and environment for reinforcement learning.
- In MDPs, the environment is fully observable.
 - Given the current state, I can forget about the history.
- Many RL problems can be formalized as MDPs:
 - In applications, we generally relax the Markov assumption
 - Formally, partially observable problems can be converted into MDPs

The Markov Property

- A state is Markov iff:

$$P[s_{t+1}|s_t] = P[s_{t+1}|s_1, s_2, \dots, s_t]$$

- Intuitive definition: *once the current state is known, you can forget about the past.*
- Formal definition: *the state is sufficient statistic of the future.*

State Transition Matrix

- For a Markov state s and a successor state s' , the state transition probability is defined by:

$$\mathcal{P}_{ss'} = \mathbb{P} [S_{t+1} = s' \mid S_t = s]$$

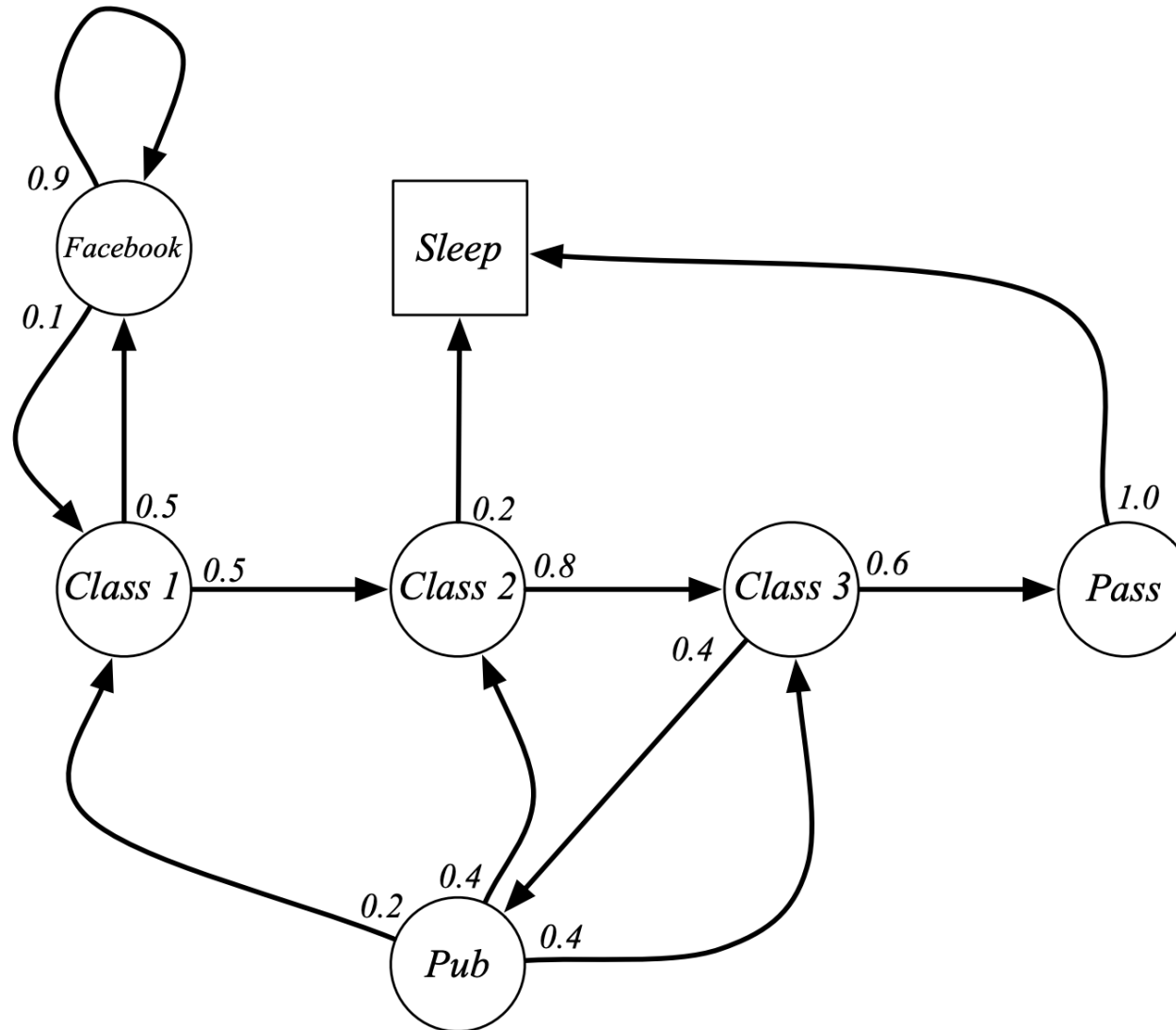
- If the environment is discrete (i.e., the number of states is finite), we can define the following state transition matrix

$$\mathcal{P} = \begin{matrix} & \text{to} \\ \text{from} & \begin{bmatrix} \mathcal{P}_{11} & \dots & \mathcal{P}_{1n} \\ \vdots & & \\ \mathcal{P}_{n1} & \dots & \mathcal{P}_{nn} \end{bmatrix} \end{matrix} \longrightarrow \text{Each row sums to one}$$

Markov Process or Chain

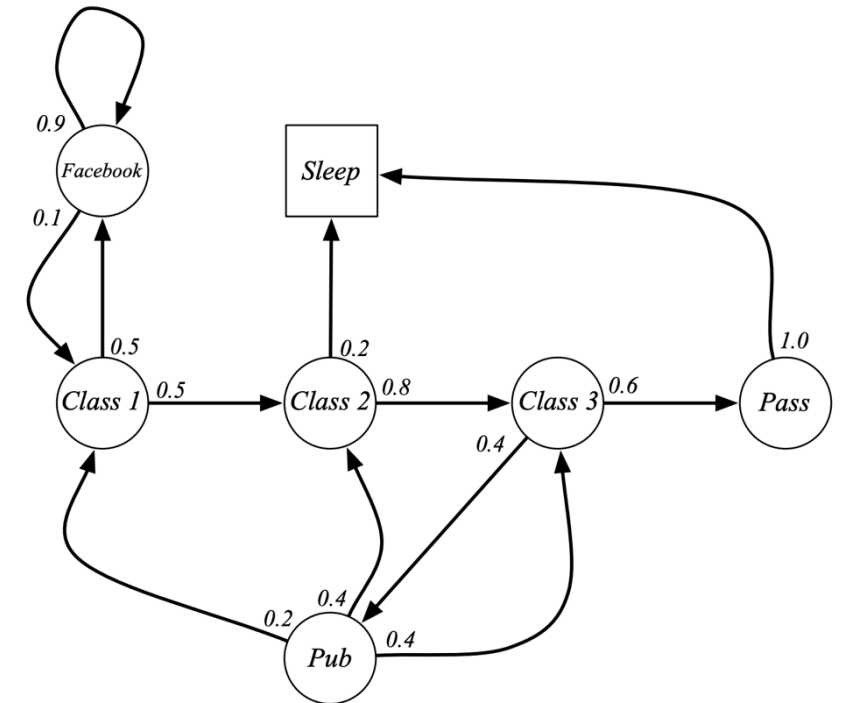
- A Markov process is a *memoryless* random process: a sequence of random states $s_1, s_2, \dots$ with the Markov property.
- Formally, a Markov process (or Markov Chain) is a tuple $\langle S, P \rangle$, where:
 - S is a (finite) set of states.
 - P is a state transition probability (matrix).

Example: Student Markov Chain

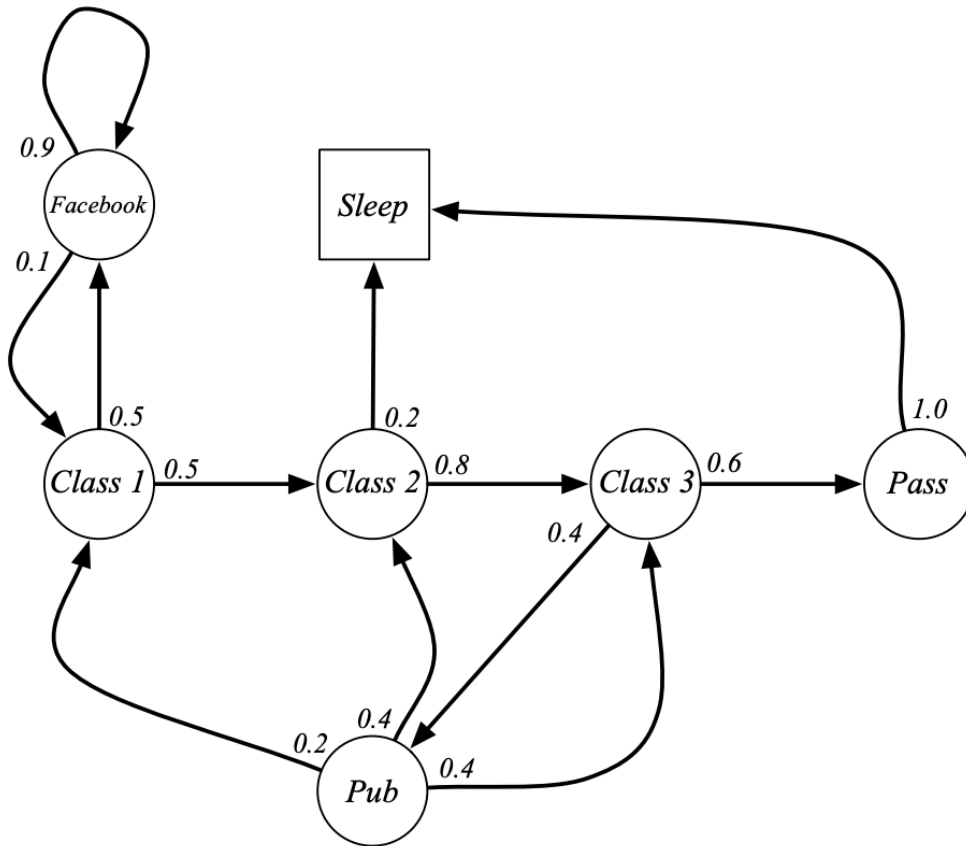


Markov Chain Episodes

- **Episodes** are a collection of states $s_0, s_1, \dots, s_T$, starting from a distribution of initial states $s_0 \sim P(s_0)$.
- Example from student Markov Chain, with $s_0 = C1$
 - C1 C2 C3 Pass Sleep
 - C1 FB FB C1 C2 Sleep
 - C1 C2 C3 Pub C2 C3 Pass Sleep
 - C1 FB FB C1 C2 C3 Pub C1 FB FB
FB C1 C2 C3 Pub C2 Sleep



Example: Transition Matrix of Student Markov Chain

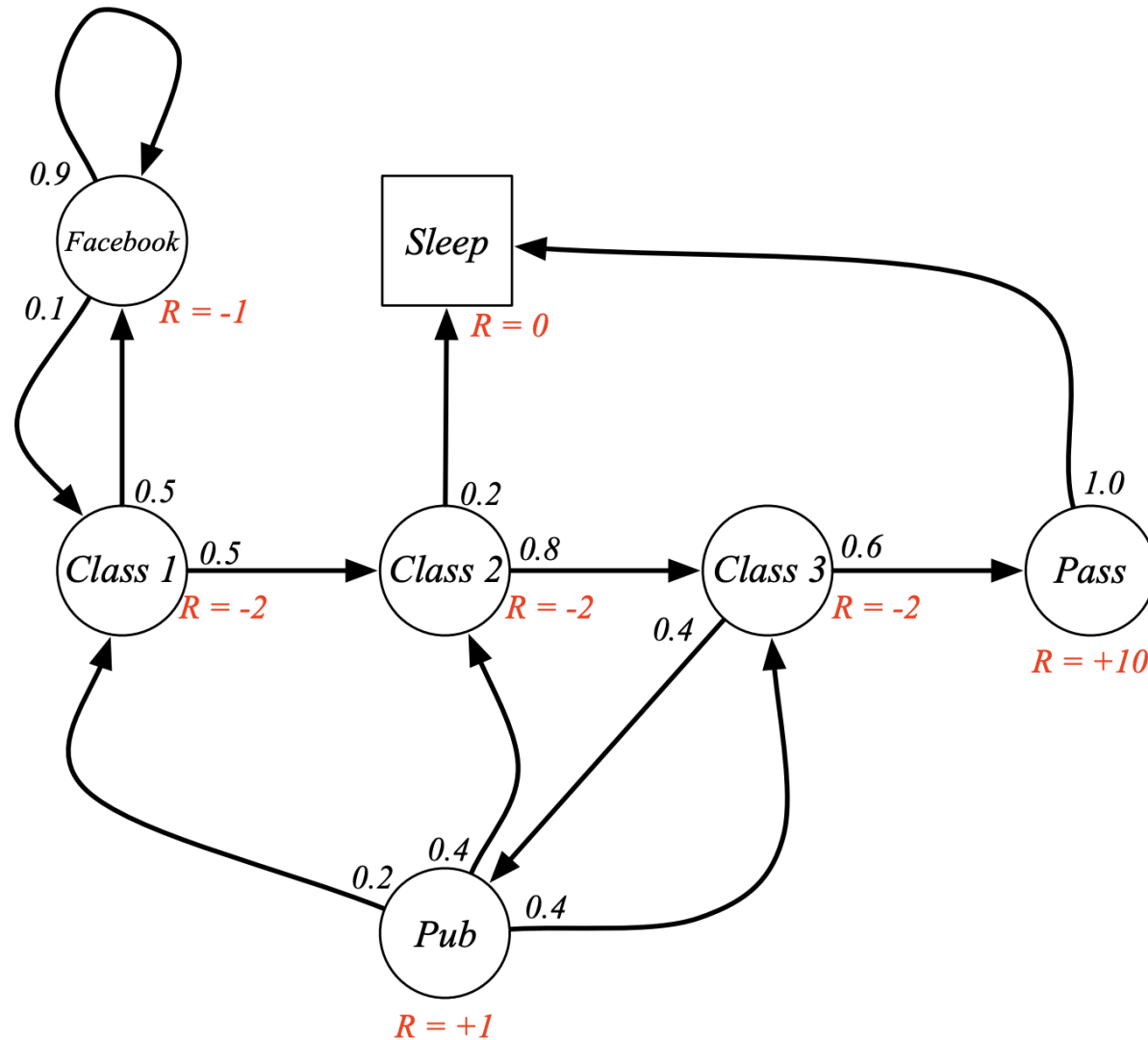


$$\mathcal{P} = \begin{matrix} & \begin{matrix} C1 & C2 & C3 & Pass & Pub & FB & Sleep \end{matrix} \\ \begin{matrix} C1 \\ C2 \\ C3 \\ Pass \\ Pub \\ FB \\ Sleep \end{matrix} & \begin{bmatrix} & 0.5 & & & & 0.5 & \\ & & 0.8 & & & & 0.2 \\ & & & 0.6 & 0.4 & & \\ 0.2 & 0.4 & 0.4 & & & & 1.0 \\ 0.1 & & & & & 0.9 & \\ & & & & & & 1 \end{bmatrix} \end{matrix}$$

Markov Reward Process

- A Markov reward process is a Markov chain with *rewards*.
- Formally, a Markov Reward Process is a tuple $\langle S, P, R, \gamma \rangle$, where
 - S is a (finite) set of states.
 - P is a state transition probability matrix.
 - R is a reward function
 - γ is a discount factor, $\gamma \in [0,1]$

Example: Student MRP



Return

- The return G_t is the cumulative discounted reward from time-step t .

$$G_t = r_{t+1} + \gamma r_{t+2} + \dots = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}$$

- The discount $\gamma \in [0,1]$ is the present value of future rewards
- The value of receiving reward r after $k + 1$ time-steps is $\gamma^k r$
- We value immediate rewards more than delayed rewards.
 - γ close to 0 leads to “myopic” evaluation
 - γ close to 1 leads to “far-sighted” evaluation
 - We will see in future lectures why $\gamma = 1$ is not always a good idea.

Why discount?

- Mathematically convenient: avoids infinite returns in cyclic Markov Processes.
- It is a simple way to represent uncertainty about the future.
- Animal/human behavior shows preference for immediate rewards
- It is sometimes possible to use *undiscounted* MRPs, with $\gamma = 1$, if all sequences terminate

The Effective Planning Horizon

- Even if episodes are very long (or infinite) , a discount factor $0 \leq \gamma < 1$ effectively reduces their length.
- Example: assume $r_t \in [-B, B]$. The return G_t is bounded by:

$$G_t \leq \sum_{k=0}^{\infty} \gamma^k B = \frac{B}{1-\gamma}$$

- This is equivalent for an *undiscounted* MRP with horizon $T = \frac{1}{1-\gamma}$.
- In general, we define $T = \frac{1}{1-\gamma}$ as the effective planning horizon of an MRP with discount factor γ .

Value Function

- The value function $V(s)$ represents the long-term value of state s .
- Formally, the state value function is the expected return starting from state s .

$$V(s) = \mathbb{E}[G_t | s_t = s]$$

Example: Student MRP Returns

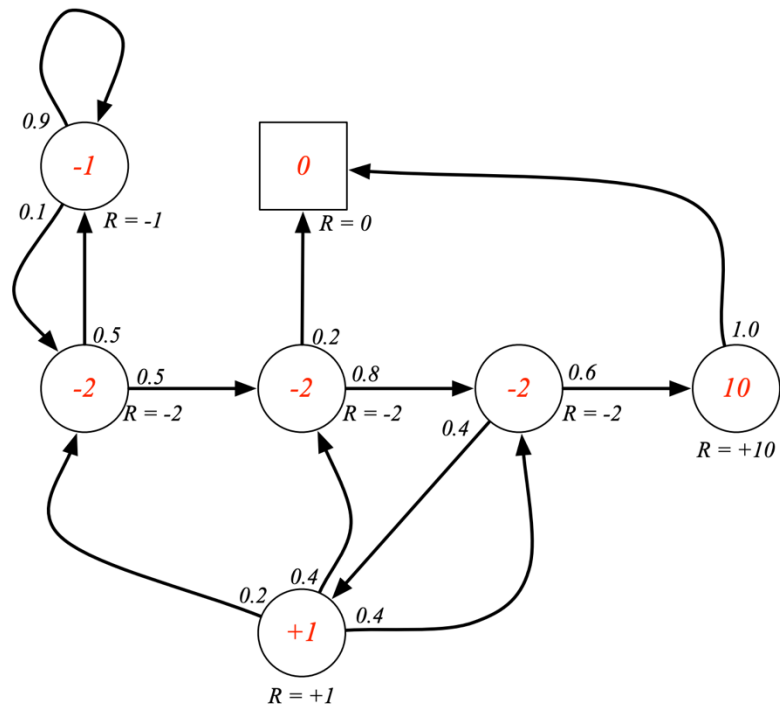
- Sample returns for the student MRP, starting from $s_1 = C1$ and $\gamma = \frac{1}{2}$.

$$G_1 = r_2 + \gamma r_3 + \dots + \gamma^{T-2} r_T$$

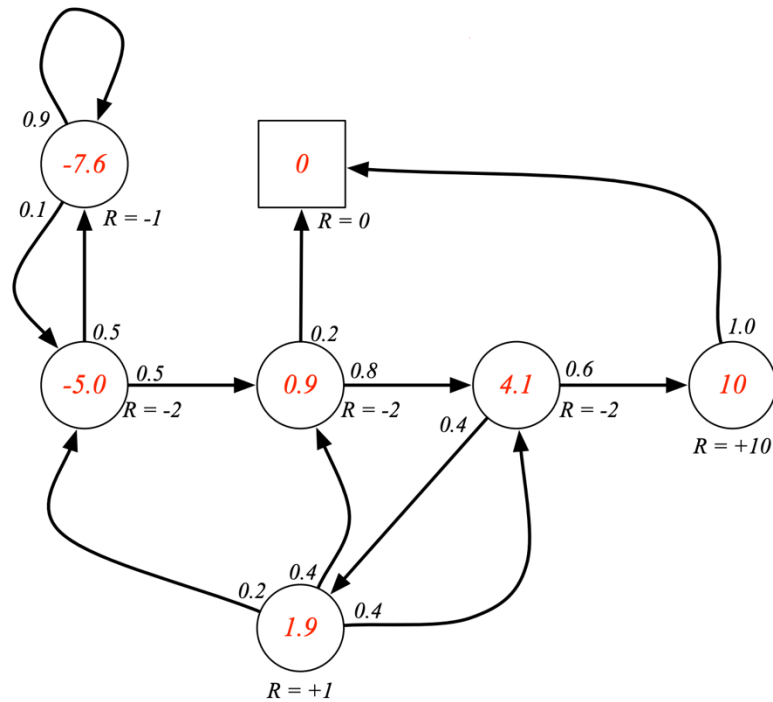
C1 C2 C3 Pass Sleep	$G_1 = -2 - 2 * \frac{1}{2} - 2 * \frac{1}{4} + 10 * \frac{1}{8}$	=	-2.25
C1 FB FB C1 C2 Sleep	$G_1 = -2 - 1 * \frac{1}{2} - 1 * \frac{1}{4} - 2 * \frac{1}{8} - 2 * \frac{1}{16}$	=	-3.125
C1 C2 C3 Pub C2 C3 Pass Sleep	$G_1 = -2 - 2 * \frac{1}{2} - 2 * \frac{1}{4} + 1 * \frac{1}{8} - 2 * \frac{1}{16} \dots$	=	-3.41
C1 FB FB C1 C2 C3 Pub C1 ...	$G_1 = -2 - 1 * \frac{1}{2} - 1 * \frac{1}{4} - 2 * \frac{1}{8} - 2 * \frac{1}{16} \dots$	=	-3.20

$$V(C1) = \text{Average } G_1 = -2.99$$

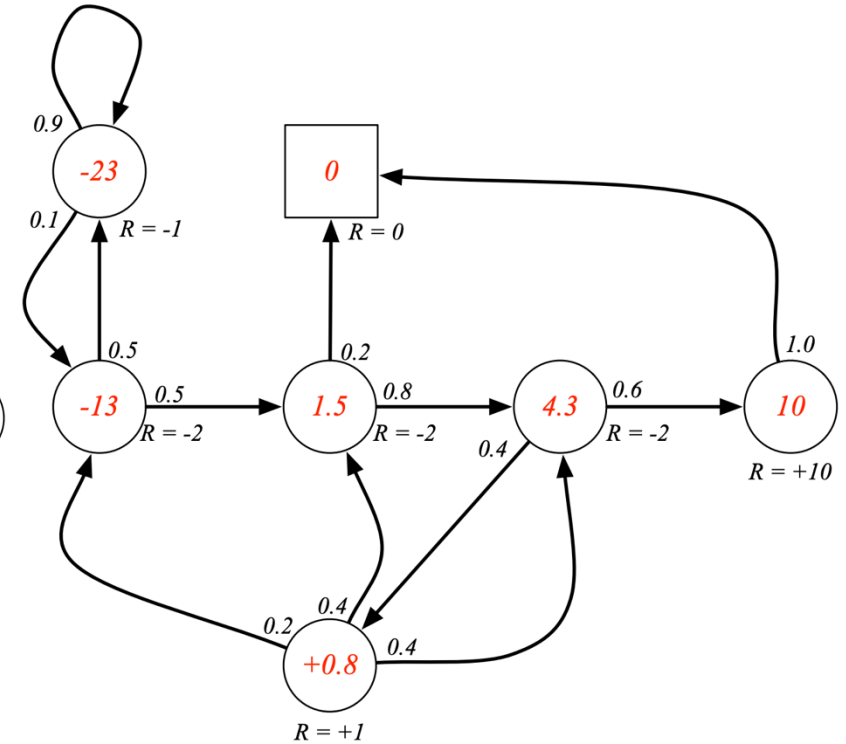
Discount Factor and Value Function: Example



$\gamma = 0$



$\gamma = 0.9$



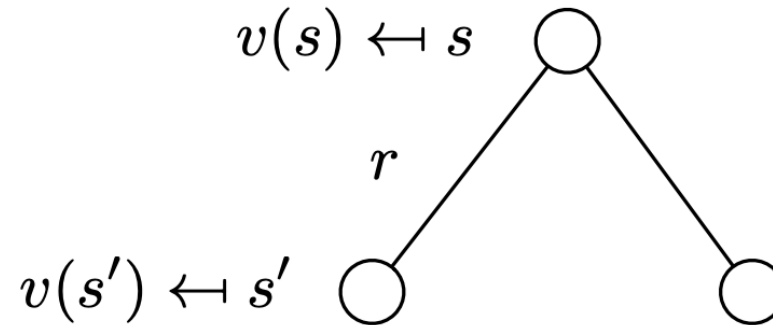
$\gamma = 1$

Bellman Equation for MRPs

- The value function can be decomposed into two parts:
 - Immediate reward r_{t+1}
 - Discounted value of successor state $\gamma V(s_{t+1})$
- $V(s) = \mathbb{E}[G_t | s_t = s]$
- $V(s) = \mathbb{E}[r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \dots | s_t = s]$
- $V(s) = \mathbb{E}[r_{t+1} + \gamma(r_{t+2} + \gamma r_{t+3} + \dots) | s_t = s]$
- $V(s) = \mathbb{E}[r_{t+1} + \gamma V(s_{t+1}) | s_t = s]$
- $V(s) = r_{t+1} + \gamma \mathbb{E}_{\mathbb{P}(s_{t+1}|s)}[V(s_{t+1}) | s_t = s]$

Bellman Equation for MRPs: Graph Representation

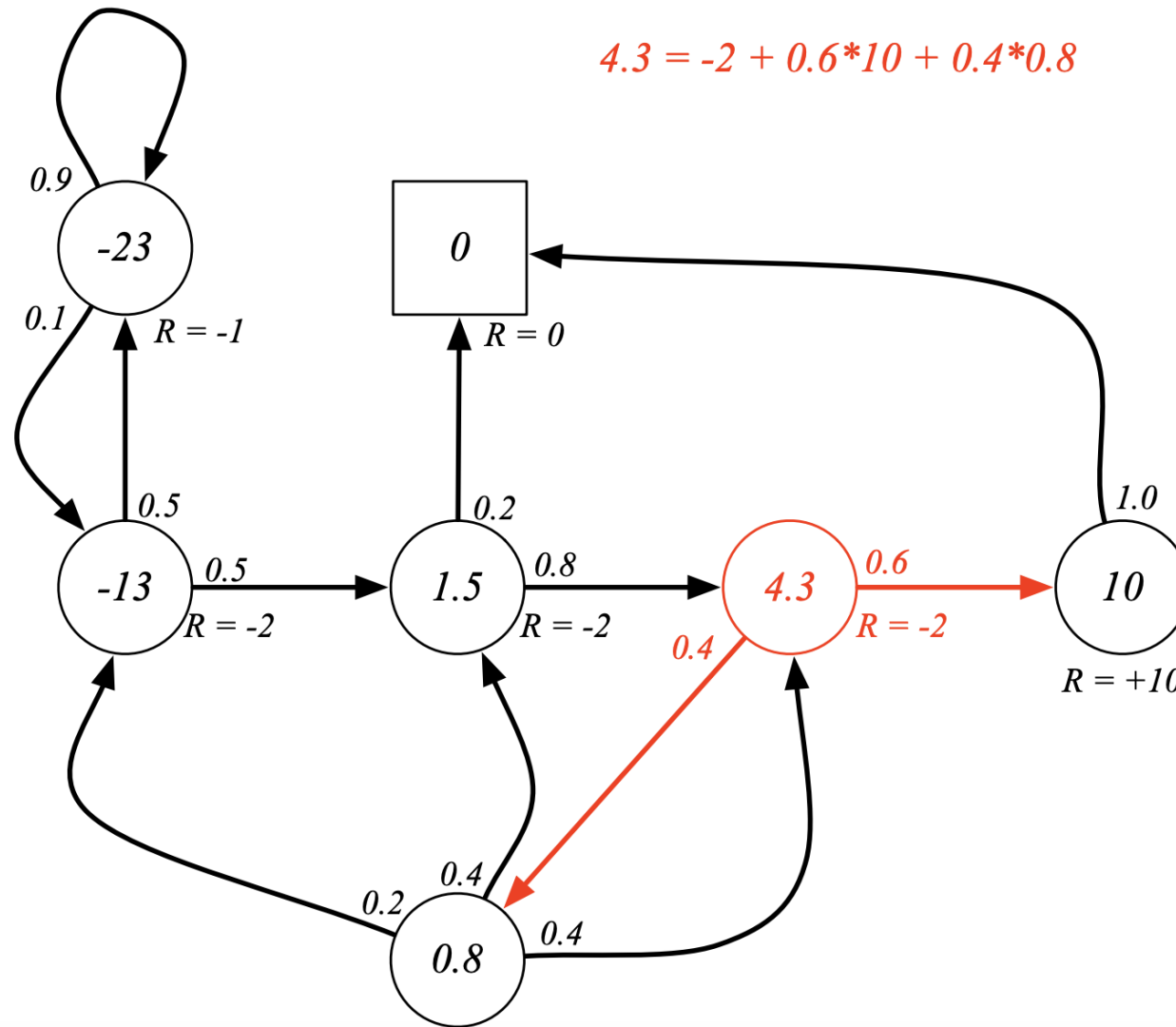
- $V^\pi(s) = \mathbb{E}[r_{t+1} + \gamma V^\pi(s_{t+1}) | s_t = s]$



- Average next-state values according to their visitation probabilities:

$$V(s) = r + \gamma \sum_{s'} P_{ss'} V(s')$$

Example: Bellman Equation for Student MRP



Bellman Equation in Matrix Form

- For discrete MRPs, the Bellman equation can be expressed concisely using matrices:

$$V = R + \gamma PV$$

Where V is a column vector with one entry per state

$$\begin{bmatrix} v(1) \\ \vdots \\ v(n) \end{bmatrix} = \begin{bmatrix} \mathcal{R}_1 \\ \vdots \\ \mathcal{R}_n \end{bmatrix} + \gamma \begin{bmatrix} \mathcal{P}_{11} & \dots & \mathcal{P}_{1n} \\ \vdots & & \\ \mathcal{P}_{n1} & \dots & \mathcal{P}_{nn} \end{bmatrix} \begin{bmatrix} v(1) \\ \vdots \\ v(n) \end{bmatrix}$$

Solving the Bellman Equation

- The Bellman equation is a *linear equation*
- It can be solved directly

$$v = \mathcal{R} + \gamma \mathcal{P} v$$

$$(I - \gamma \mathcal{P}) v = \mathcal{R}$$

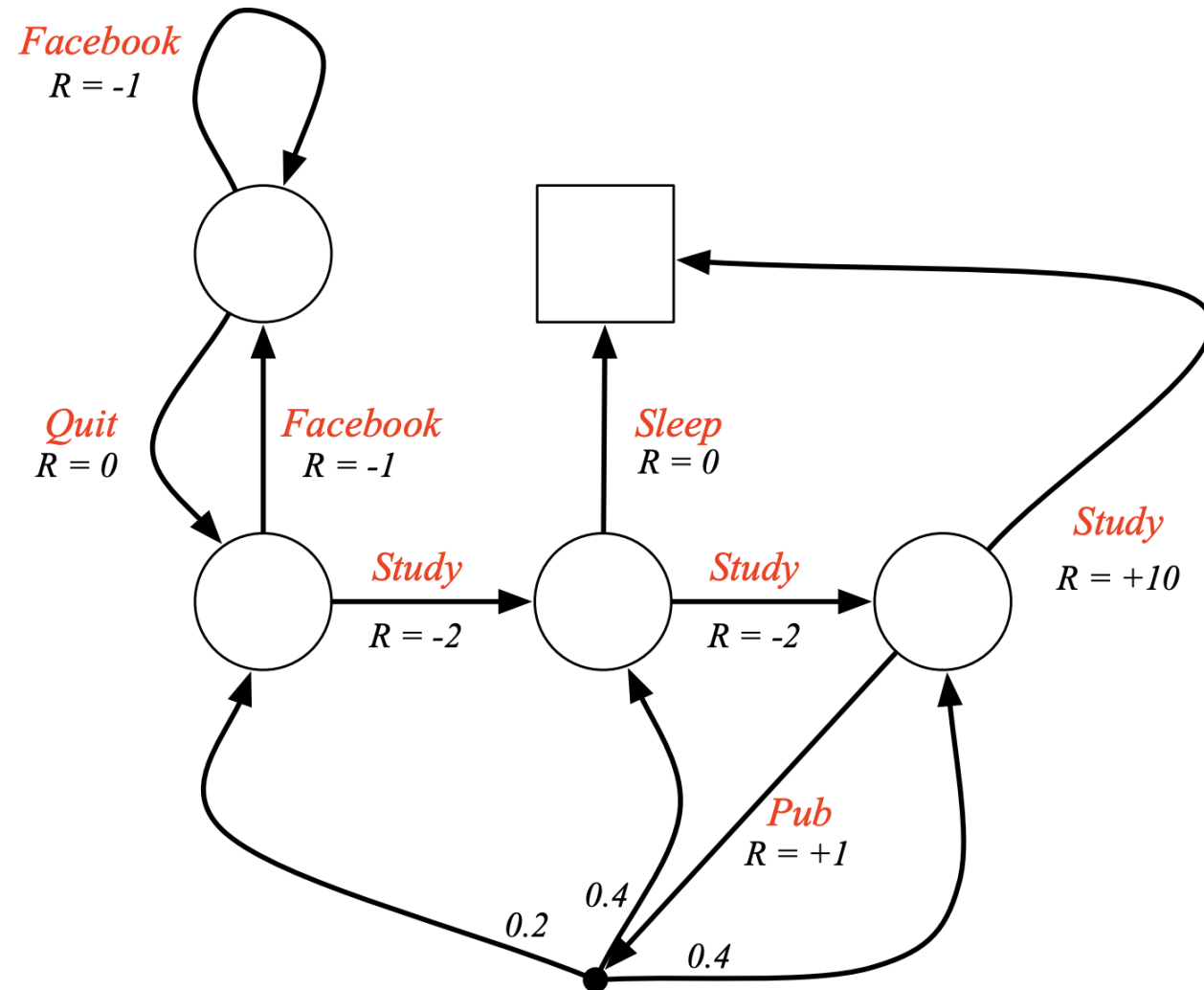
$$v = (I - \gamma \mathcal{P})^{-1} \mathcal{R}$$

- Simple! Why is this often not possible in real problems?

Markov Decision Process

- A Markov Decision Process (MDP) is a Markov reward process with decisions. It is an *environment* in which all states are Markov.
- Formally, a Markov Decision Process is a tuple $\langle S, A, P, R, \gamma \rangle$, where
 - S is a (finite) set of states.
 - A is the set of actions
 - P is a state transition probability matrix, with elements
$$P_{ss'}^a = \mathbb{P}[s_{t+1} = s' | s_t = s, a_t = a]$$
 - R is a reward function
 - γ is a discount factor, $\gamma \in [0,1]$

Example: Student MDP



Policies

- A policy π is a distribution of over actions given states

$$\pi(a|s) = \mathbb{P}[a_t = a | s_t = s]$$

- A policy defines the behavior of an agent
- Policies in MDPs depend on the *current state* (not the history)
- And therefore, they are time-independent (i.e., stationary)

$$a_t \sim \pi(\cdot | s_t), \forall t > 0$$

Policies

- Given an MDP $M = \langle S, \textcolor{red}{A}, P, R, \gamma \rangle$ and a policy π
- The *state sequence* $s_1, s_2, \dots$ is a Markov process $\langle S, P^\pi \rangle$
- The state and reward sequence $s_1, r_1, s_2, r_2, \dots$ is a Markov reward process $\langle S, P^\pi, R^\pi, \gamma \rangle$, where

$$P_{ss'}^\pi = \sum_{a \in A} \pi(a|s) P_{ss'}^a$$
$$R_s^\pi = \sum_{a \in A} \pi(a|s) R_s^a$$

Value Function

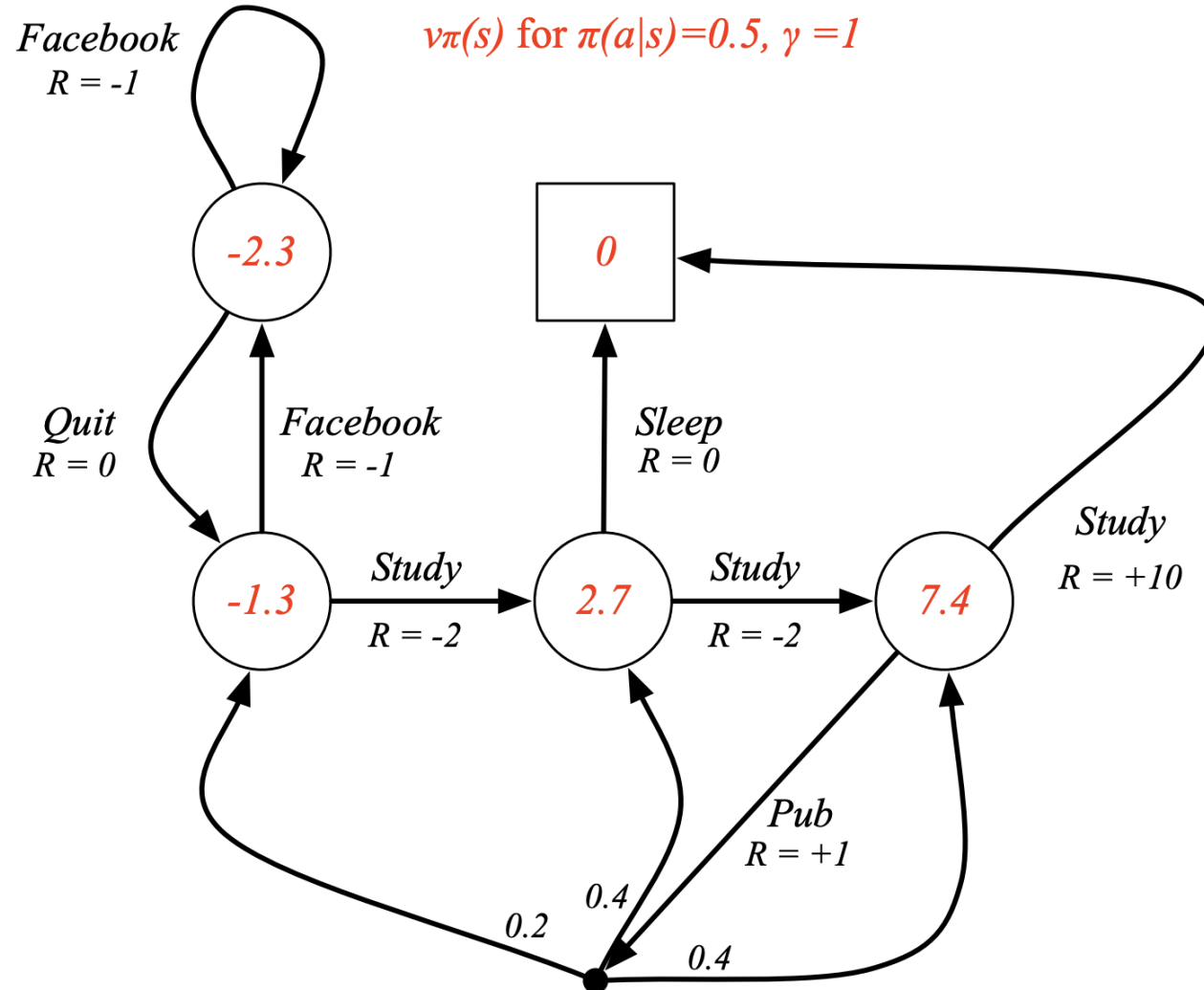
- The **state-value function** $V_\pi(s)$ of an MDP is the expected return starting from state s , and then following policy π

$$V^\pi(s) = \mathbb{E}_\pi[G_t | s_t = s]$$

- The **action-value function (or Q function)** $Q_\pi(s, a)$ is the expected return starting from state s , taking action a , and then following π .

$$Q^\pi(s, a) = \mathbb{E}_\pi[G_t | s_t = s, a_t = a]$$

Example: State-Value Function for Student MDP



Bellman Expectation Equation

- The state-value function can again be decomposed into immediate reward plus discounted value of successor state:

$$V^{\pi}(s) = \mathbb{E}_{\pi}[r_{t+1} + \gamma V^{\pi}(s_{t+1}) | s_t = s]$$

- The action-value function can similarly be decomposed

$$Q^{\pi}(s, a) = \mathbb{E}_{\pi}[r_{t+1} + \gamma Q^{\pi}(s_{t+1}, a_{t+1}) | s_t = s, a_t = a]$$

Value and Q Function Relationship

- The state-value function $V_\pi(s)$:

$$V^\pi(s) = \mathbb{E}_\pi[G_t | s_t = s]$$

- The action-value function (or Q function) $Q_\pi(s, a)$:

$$Q^\pi(s, a) = \mathbb{E}_\pi[G_t | s_t = s, a_t = a]$$

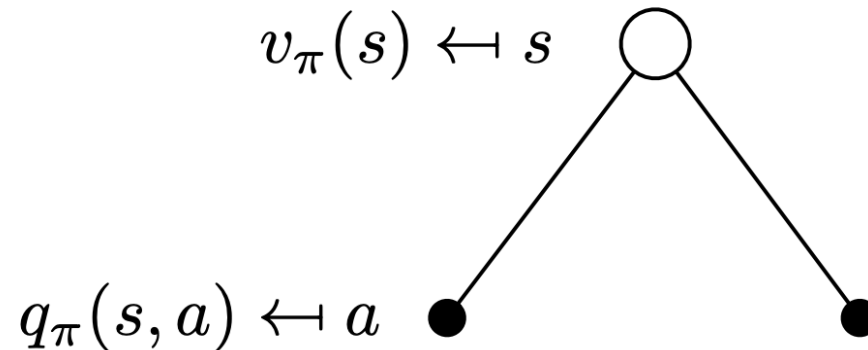
- What is $Q^\pi(s, \pi(s)) = \mathbb{E}_\pi[G_t | s_t = s, a_t = \pi(\cdot | s)]$?

Bellman Expectation Equation: Graph Representation

- The Q and State-Value functions are related:

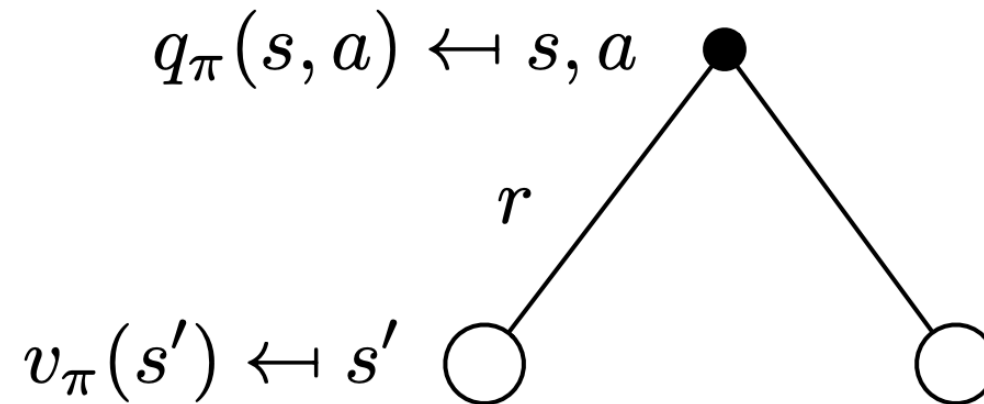
$$V^\pi(s) = E_\pi[Q^\pi(s_t, a_t) | s_t = s, a_t = \pi(\cdot | s)]$$

- For discrete state-action spaces, $V_\pi(s) = \sum_{a \in A} \pi(a|s) Q_\pi(s, a)$



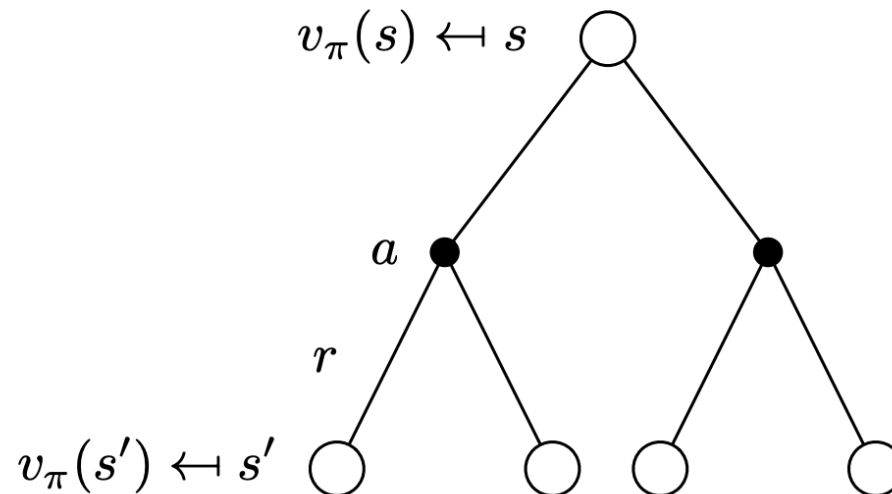
Bellman Expectation for Q^π

$$Q^\pi(s, a) = r(s, a) + \gamma \sum_{s'} P_{ss'}^a V^\pi(s')$$



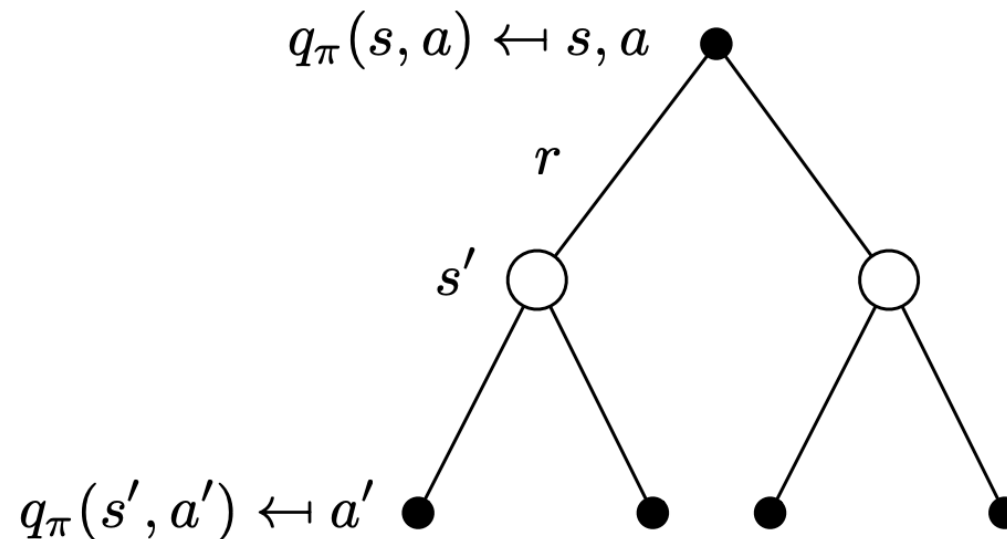
Bellman Expectation Equation for V^π

$$V^\pi(s) = \sum_{a \in A} \pi(a|s)(r(s, a) + \gamma \sum_{s'} P_{ss'}^a V^\pi(s'))$$

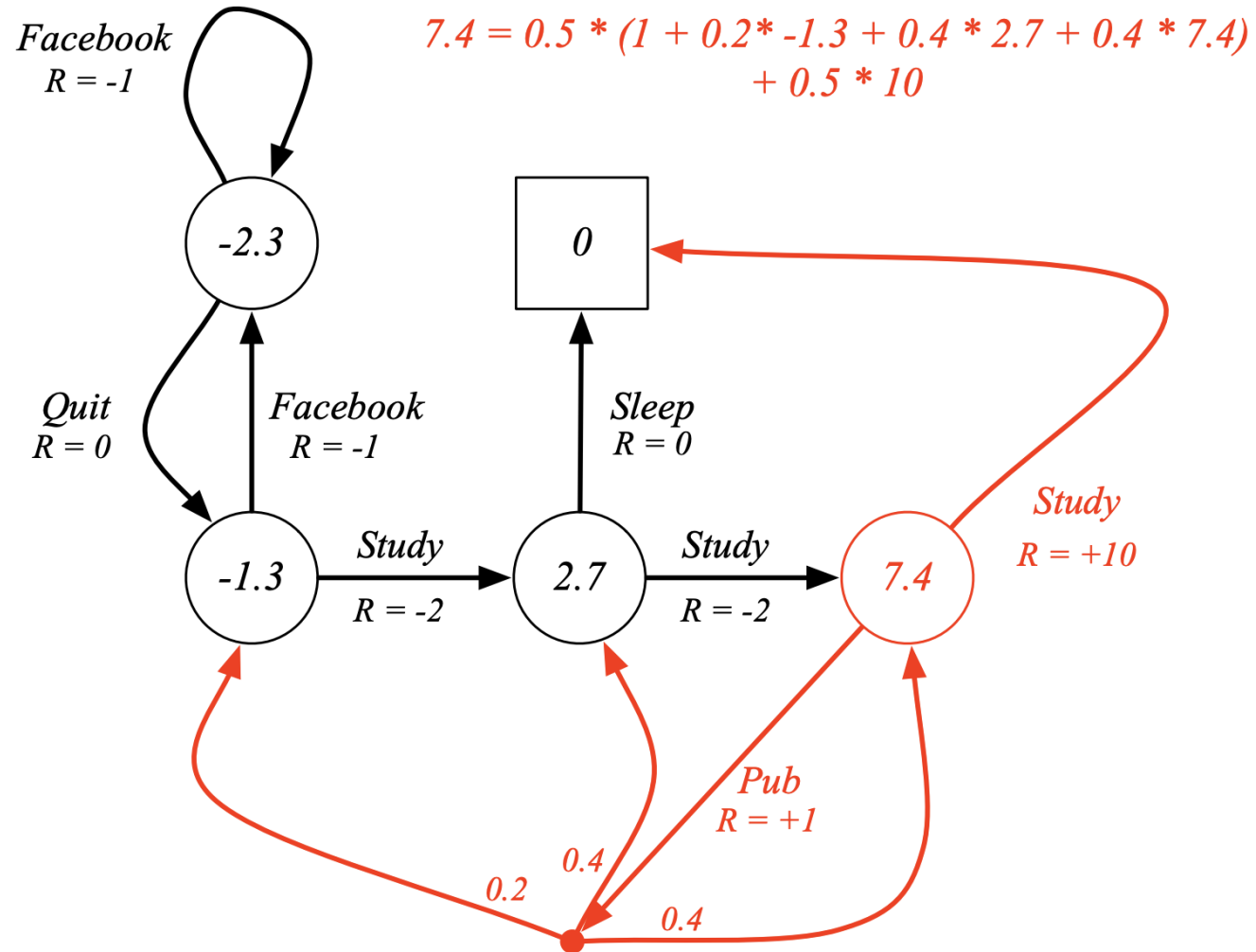


Bellman Expectation Equation for Q^π

$$Q^\pi(s, a) = r(s, a) + \gamma \sum_{s'} P_{ss'}^a \sum_{a' \in A} \pi(a'|s') Q^\pi(s', a')$$



Example: Student MDP



Bellman Expectation Equation (Matrix Form)

- We can write the Bellman expectation equation induced by the MRP concisely in matrix form

$$V^\pi = R^\pi + \gamma P^\pi V^\pi$$

- And directly compute a solution

$$V^\pi = (I - \gamma P^\pi)^{-1} R^\pi$$

Optimal Value Function

- The **optimal state-value function** $V_*(s)$ is the maximum value function over all policies

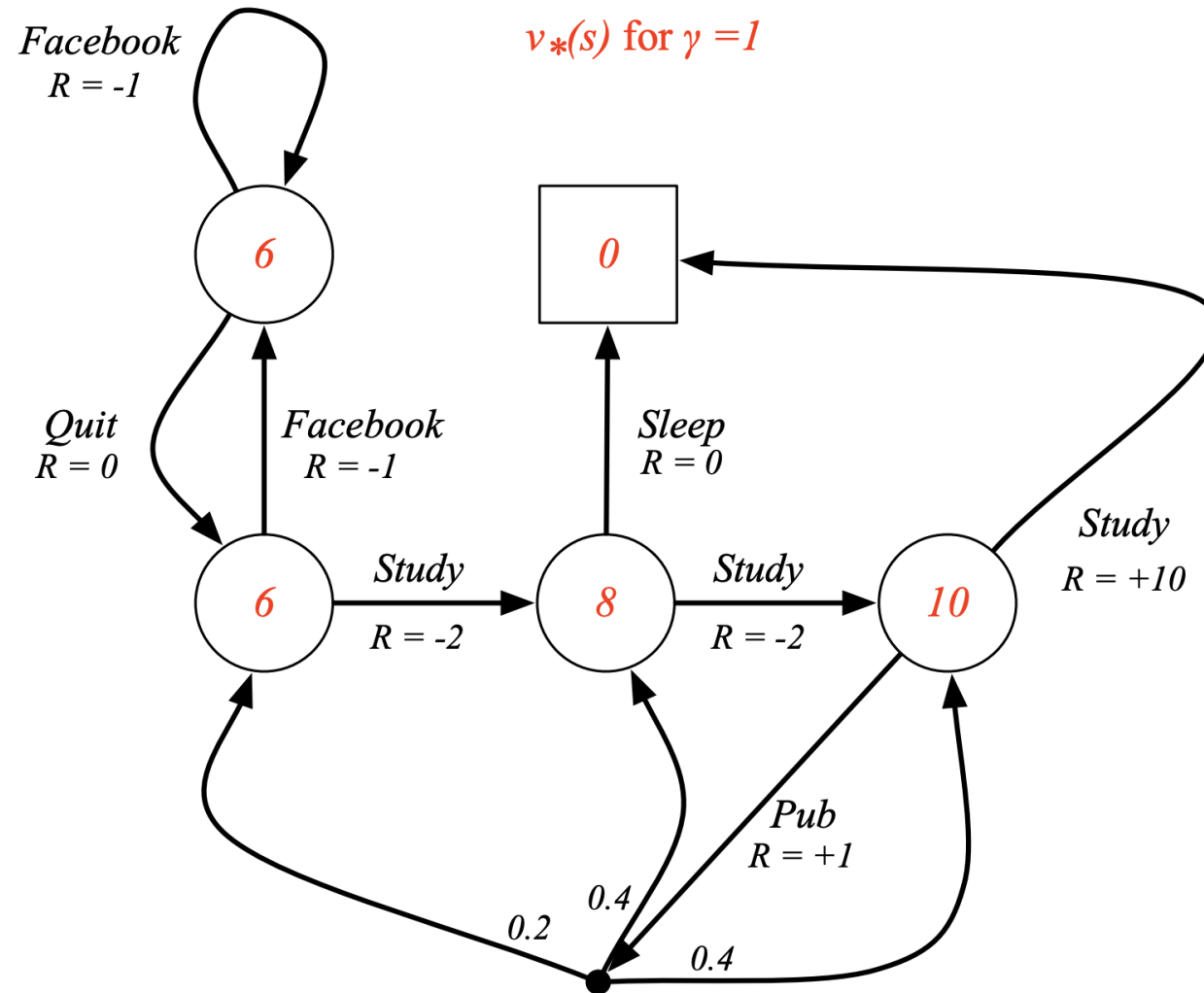
$$V_*(s) = \max_{\pi} V^{\pi}(s)$$

- The **optimal action-value function** $Q_*(s, a)$ is the maximum action-value function over all policies

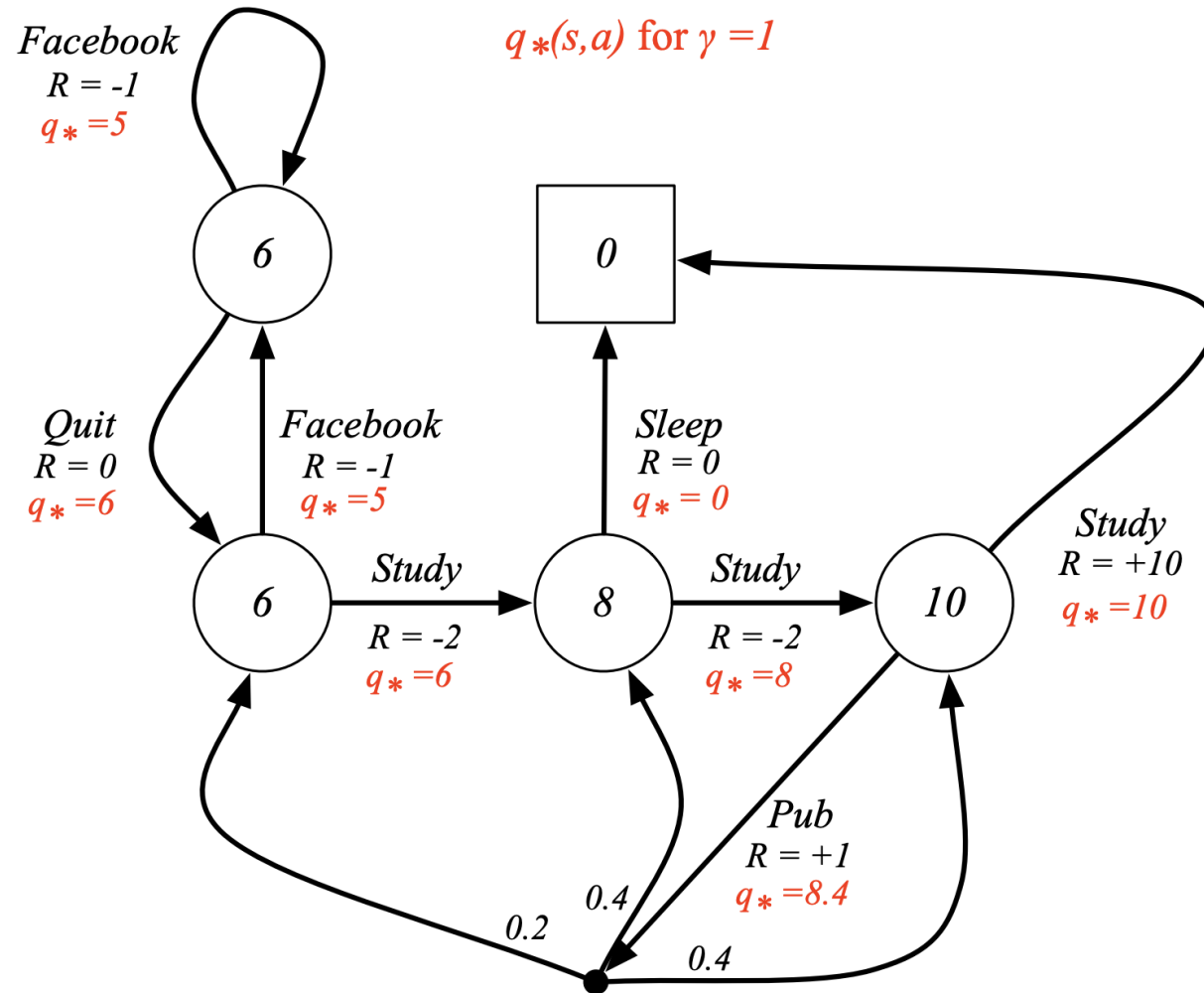
$$Q_*(s, a) = \max_{\pi} Q^{\pi}(s, a)$$

- They specify the best performance, so once we know at least one of them, the MDP is solved.

Example: Optimal Value Function for Student MDP



Example: Optimal Q function for Student MDP



Optimal Policy

- Define a partial ordering over policies: $\pi \geq \pi'$ if $V^\pi(s) \geq V^{\pi'}(s), \forall s$.
- *Theorem.* For any MDP:
 - There exists an optimal policy π_* that is better than or equal to all other policies.
 - All optimal policies achieve the optimal value function.
 - All optimal policies achieve the optimal action-value function.

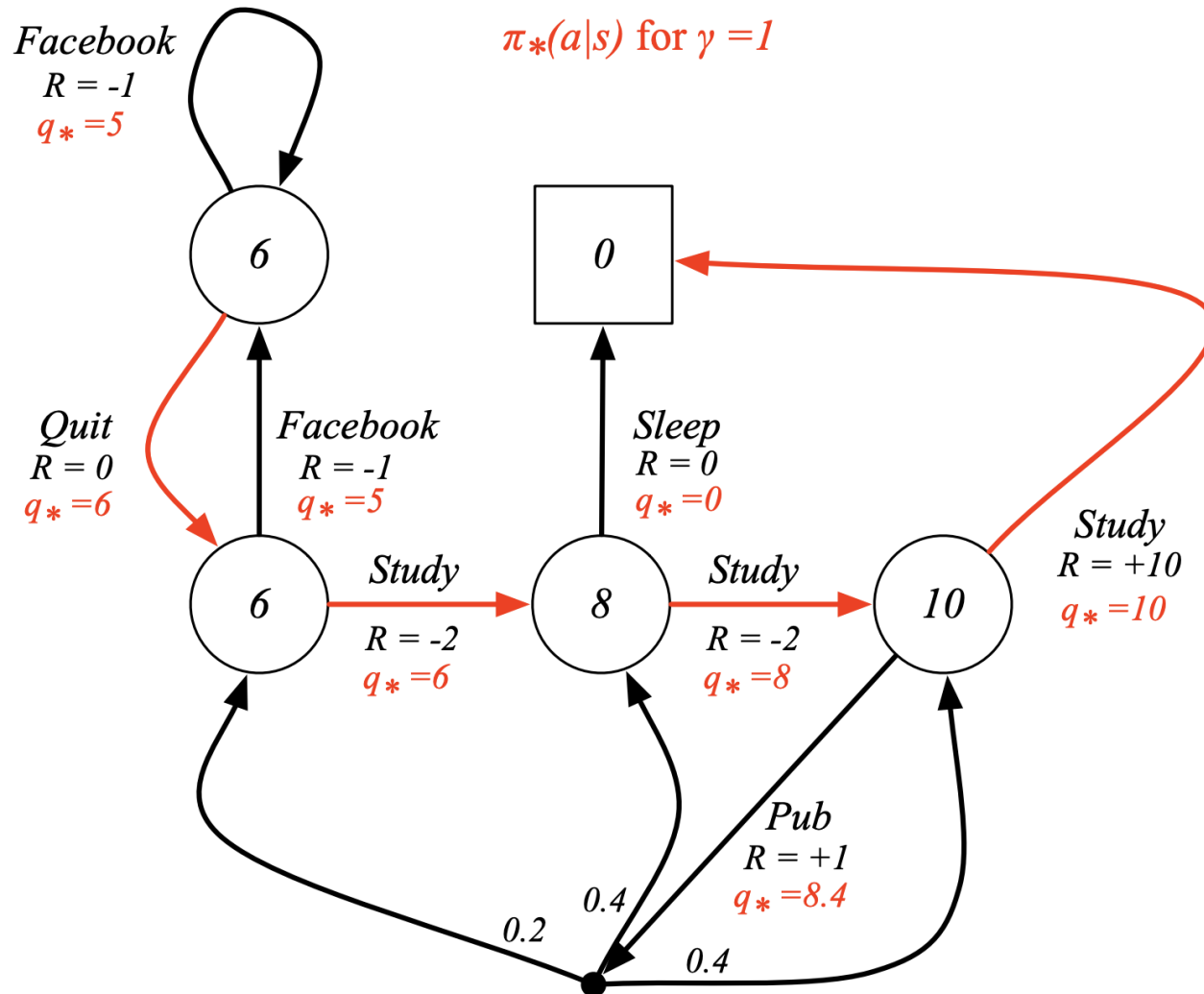
Finding an Optimal Policy

- The goal of this course.
- There is always a deterministic optimal policy for any MDP (whether or not the MDP has stochastic transition probabilities).
 - Even a crazy world can be controlled by a good plan 😊
- An optimal policy can be found by maximizing over $Q_*(s, a)$,

$$\pi_*(a|s) = \begin{cases} 1 & \text{if } a = \operatorname{argmax}_{a \in A} Q_*(s, a) \\ 0 & \text{otherwise} \end{cases}$$

- If we know the $Q_*(s, a)$, we immediately have the optimal policy

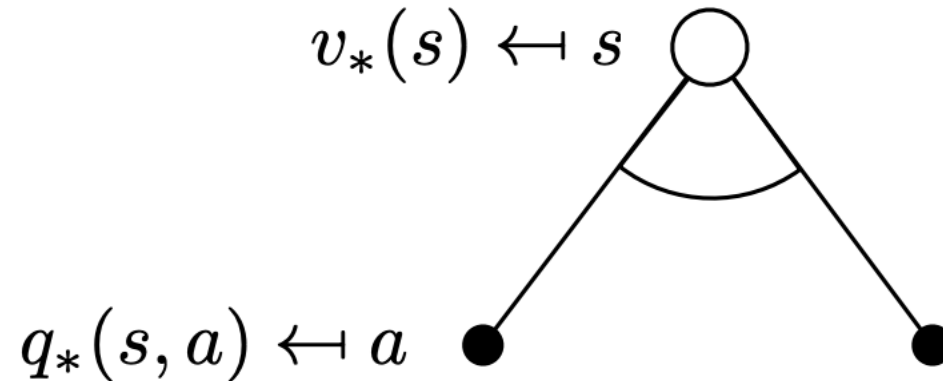
Example: Optimal Policy for Student MDP



Bellman Optimality Equation for V_*

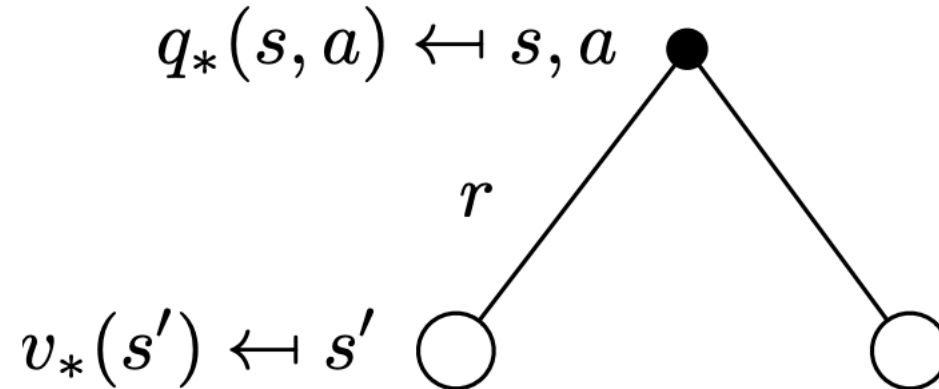
- The optimal value functions are recursively related by the Bellman optimality equations:

$$V_*(s) = \max_a Q_*(s, a)$$



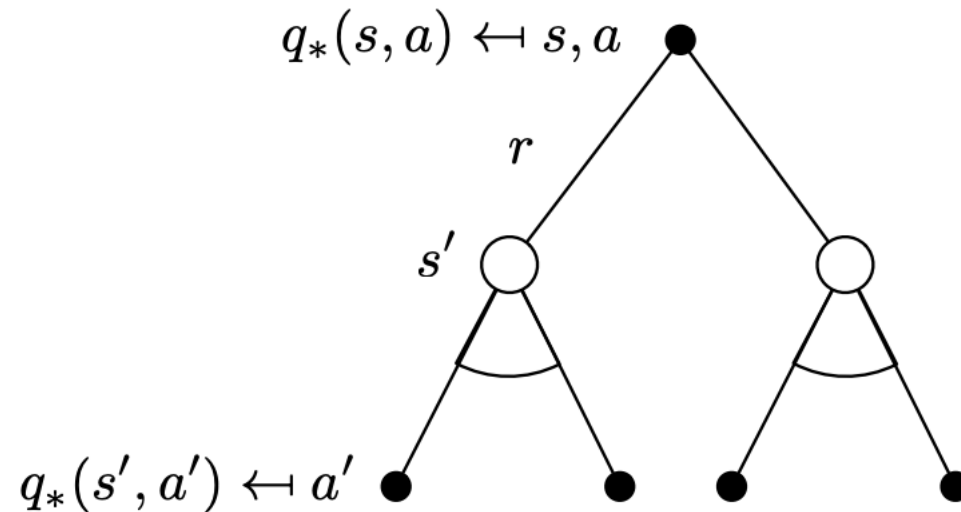
Bellman Optimality Equation for Q^*

$$Q_*(s, a) = r(s, a) + \gamma \sum_{s' \in \mathcal{S}} P_{ss'}^a V_*(s')$$

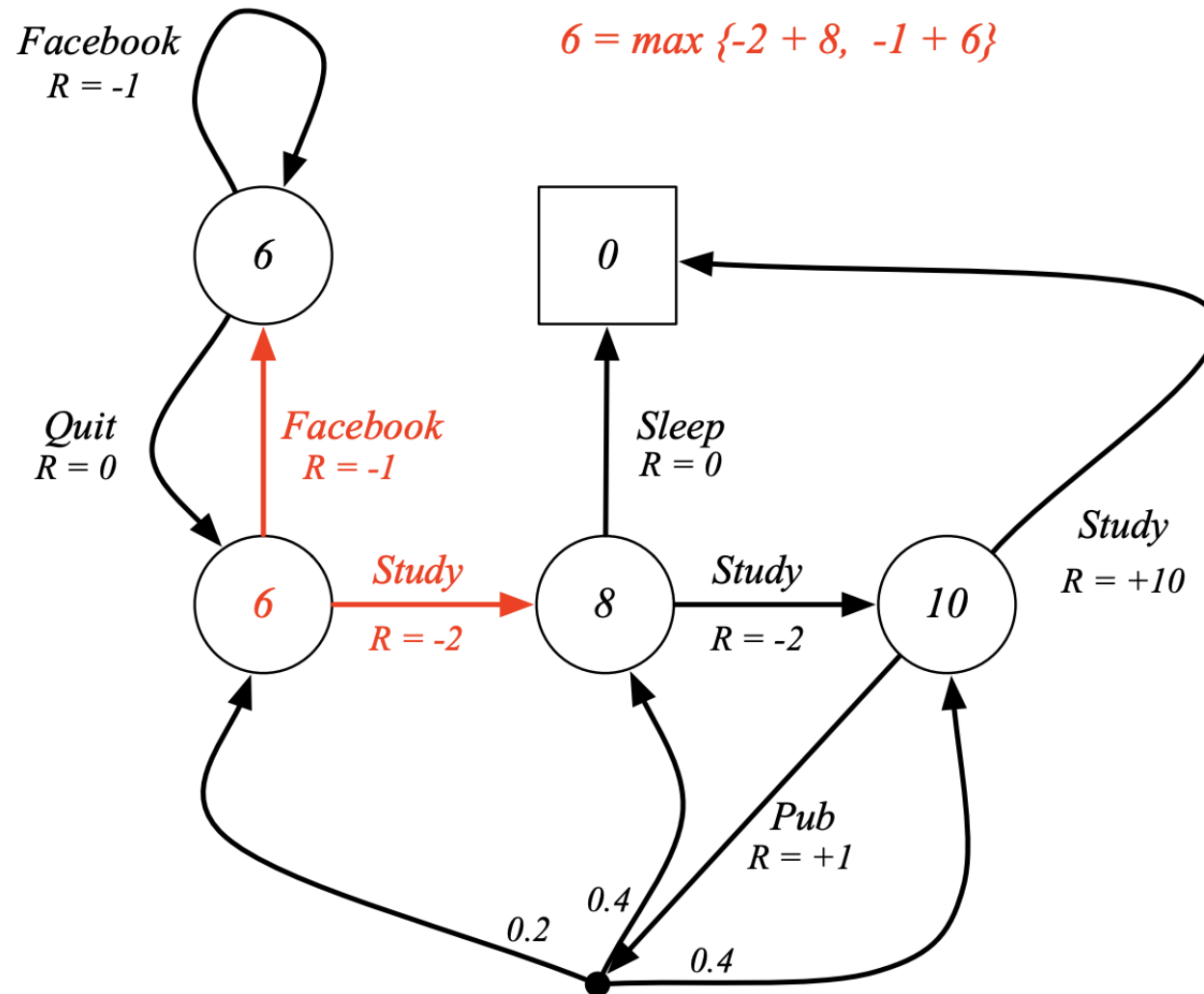


Recursive Bellman Optimality Equation for Q_*

$$Q_*(s, a) = r(s, a) + \gamma \sum_{s' \in S} P_{ss'}^a \max_{a'} Q_*(s', a')$$



Example: Bellman Optimality Equation in Student MDP



Solving the Bellman Optimality Equation

- The Bellman Optimality Equation is non-linear (due to the max operation).
- No closed-form solution in general
- Many iterative solutions methods we will see in class
 - Value Iteration
 - Policy Iteration
 - Q-learning
 - Sarsa

Extensions to MDPs

- Infinite and continuous MDPs
- Partially Observable MDPs (-> most problems we care about)
- Undiscounted, average reward MDPs

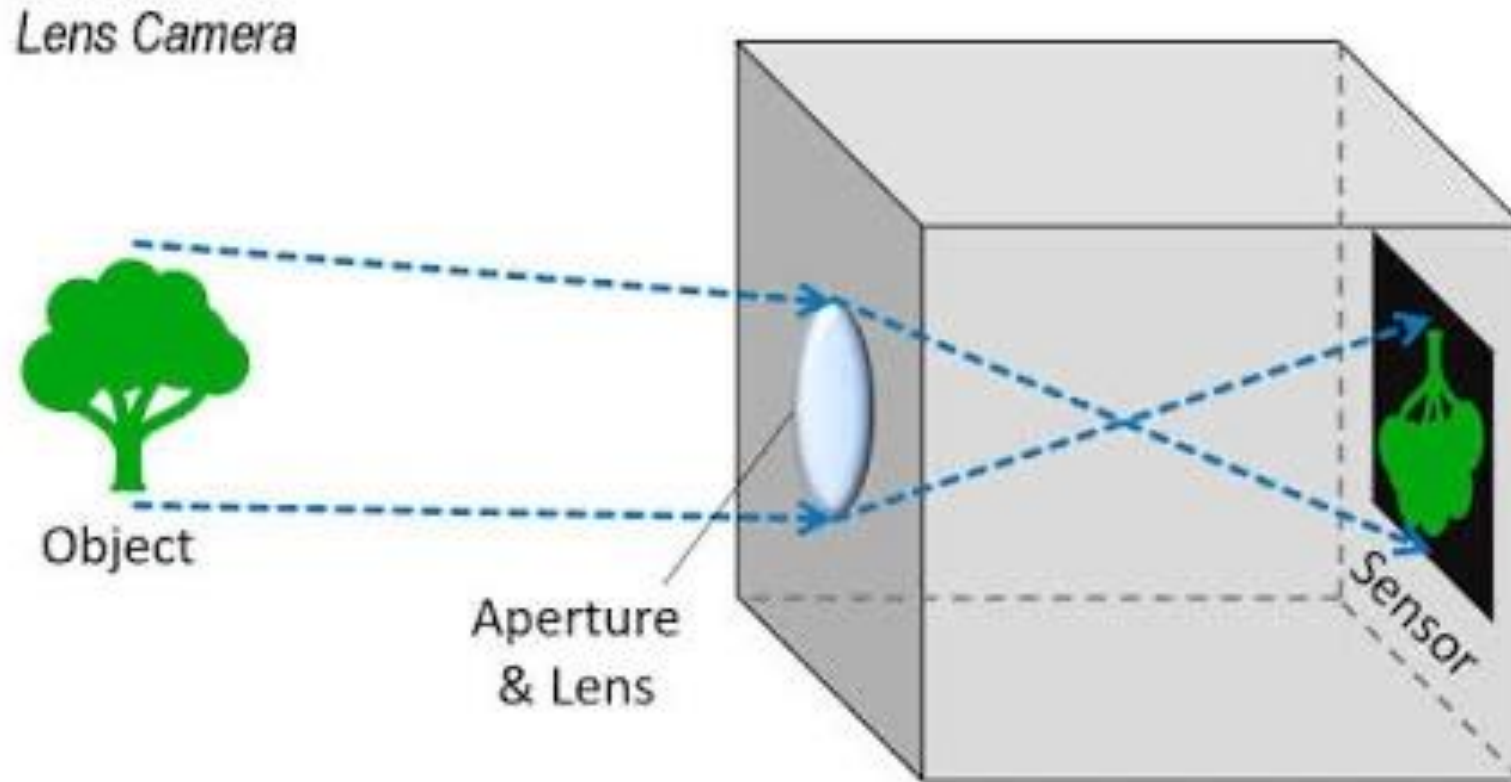
Infinite MDPs

- The following extensions are possible
- Countably infinite state and/or action spaces
 - Same equations as above
- Continuous state and/or action spaces:
 - Closed-form solutions in specific cases!!! (LQR)
- Continuous time
 - Requires partial differential equations
 - Hamilton-Jacobi-Bellman (HJB) equation -> you'll see this in control classes
 - Limiting case of Bellman equation as time-step $t \rightarrow 0$

POMDPs (a.k.a. almost any problem)

- A Partially Observable Markov Decision Process is an MDP with hidden states. It is a hidden Markov model with actions.
- Formally, it is a tuple $\langle S, A, O, P, R, Z, \gamma \rangle$, where
 - S is a (finite) set of states.
 - A is the set of actions
 - O is the finite set of observations
 - P is a state transition probability matrix, with elements
$$P_{ss'}^a = \mathbb{P}[s_{t+1} = s' | s_t = s, a_t = a]$$
 - Z is the observation function $Z_{s'o}^a = \mathbb{P}[o_{t+1} = o | s_t = s', a_t = a]$
 - R is a reward function
 - γ is a discount factor, $\gamma \in [0,1]$

Example: Observation Function for a Camera



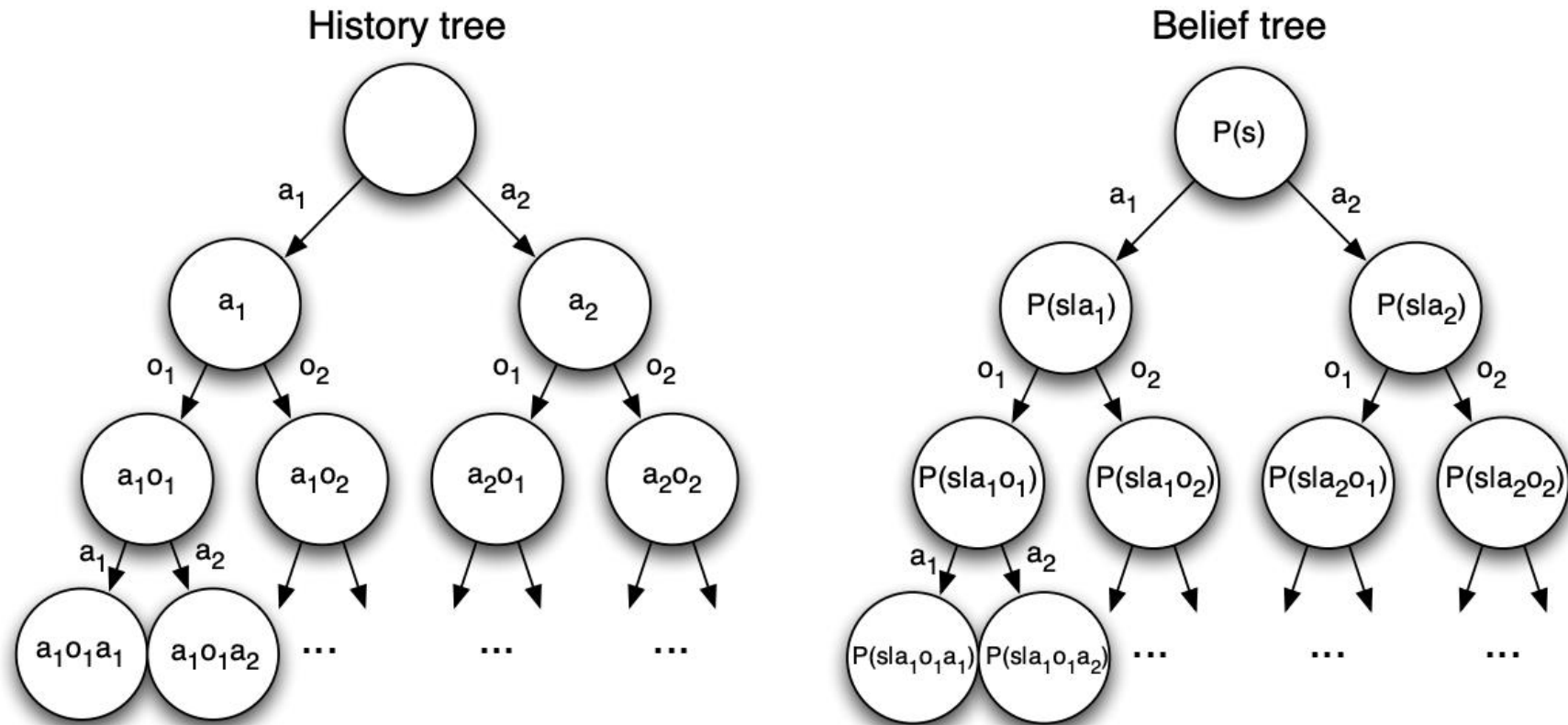
Source: Hardware Acceleration of Progressive Refinement Radiosity using Nvidia RTX, 2023

Transforming a POMDP into an MDP

- Mainly two options:
- Use the *history* $H_t = a_0, o_1, r_1, \dots, a_{t-1}, o_t, r_t$ as the MDP's state
- Use the *belief state* $b(h)$, which is a probability distribution over states conditioned on the history H , as the MDP's state.

$$b(h) = (\mathbb{P}[s_t = s_1 | H_t = h], \dots, \mathbb{P}[s_t = s^n | H_t = h])$$

Reduction of POMDPs to MDPs



- Note: In practice, hard to work with these objects.

Ergodic Markov Process

- A Markov Process is ergodic if:
 - *You can reach any state from any other state (irreducible).*
 - *Each state is visited without any specific period (aperiodic).*
- *Theorem.* An ergodic Markov process has a *limiting* stationary distribution $d^\pi(s)$ with the property

$$d^\pi(s) = \sum_{s' \in S} d^\pi(s') P_{s's}$$

- English version:
 - The long-term average behavior of a single random path matches the expected behavior of the entire system.

Ergodic MDP

- An MDP is ergodic if the Markov chain induced by *any policy* is ergodic.
- For any policy π , an ergodic MDP has an *average reward per time-step* ρ^π that is independent of start state.

$$\rho^\pi = \lim_{T \rightarrow \infty} \frac{1}{T} \mathbb{E} \left[\sum_{t=1}^T r_t \right]$$